

PROJECT REPORT

EXPERIMENT - 7

Project Statement

Design and develop a Wireless Environmental Parameter Monitoring System for Smart Industries. The system remotely monitors environmental conditions such as temperature, humidity, and air quality in industrial environments.

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

Wireless Environmental Parameter Monitoring System for Smart Industries

Project Overview

The Wireless Environmental Parameter Monitoring System for Smart Industries is an IoT-based system designed to continuously monitor important environmental conditions in industrial environments. The system uses an ESP8266 Node MCU as the main controller, a DHT11/DHT22 sensor to measure temperature and humidity, and an MQ135 sensor to detect changes in air quality.

The collected environmental data can be transmitted wirelessly through the ESP8266 Wi-Fi connection for remote monitoring. When the measured conditions exceed predefined safety limits, a buzzer provides an immediate local warning. The system helps industries identify unfavorable environmental conditions and improve workplace safety.

Main Objectives:

- To monitor temperature continuously.
- To measure relative humidity in the industrial environment.
- To detect changes in air quality using the MQ135 sensor.
- To provide wireless monitoring using ESP8266 Wi-Fi.
- To generate an alert when environmental conditions exceed set limits.
- To develop a low-cost and compact environmental monitoring prototype.
- To support preventive action in smart industrial environments.

Problem Statement

- Industrial environments may experience changes in temperature, humidity, and air quality due to machinery operation, chemical processes, ventilation problems, and other activities. Manual monitoring of these parameters can be time-consuming and may not provide continuous information.
- Therefore, a Wireless Environmental Parameter Monitoring System is required to continuously monitor temperature, humidity, and air quality and provide timely alerts when abnormal environmental conditions are detected.

Problems Addressed:

- Lack of continuous environmental monitoring.
- Delayed identification of poor air quality.
- Difficulty in manually monitoring industrial conditions.
- Lack of immediate warning during abnormal conditions.
- Limited accessibility to environmental data from remote locations.
- Need for an inexpensive and compact monitoring system.

Proposed Solution

- The proposed system uses an ESP8266 Node MCU to collect data from the DHT11/DHT22 and MQ135 sensors.
- The DHT sensor measures temperature and humidity, while the MQ135 detects changes in air quality. The ESP8266 processes the sensor readings and uses its built-in Wi-Fi capability to transmit the data wirelessly.
- If the measured parameters cross predefined threshold values, the ESP8266 activates the buzzer to provide an immediate warning.

Proposed Operation:

- The DHT11/DHT22 sensor measures temperature and humidity.
- The MQ135 sensor senses changes in air quality.
- Sensor outputs are sent to the ESP8266 Node MCU.
- The ESP8266 processes the received sensor values.
- The values are monitored against predefined threshold limits.
- Environmental data can be transmitted through Wi-Fi.
- If an abnormal condition is detected, the buzzer is activated.
- The system continues monitoring continuously.

System Architecture

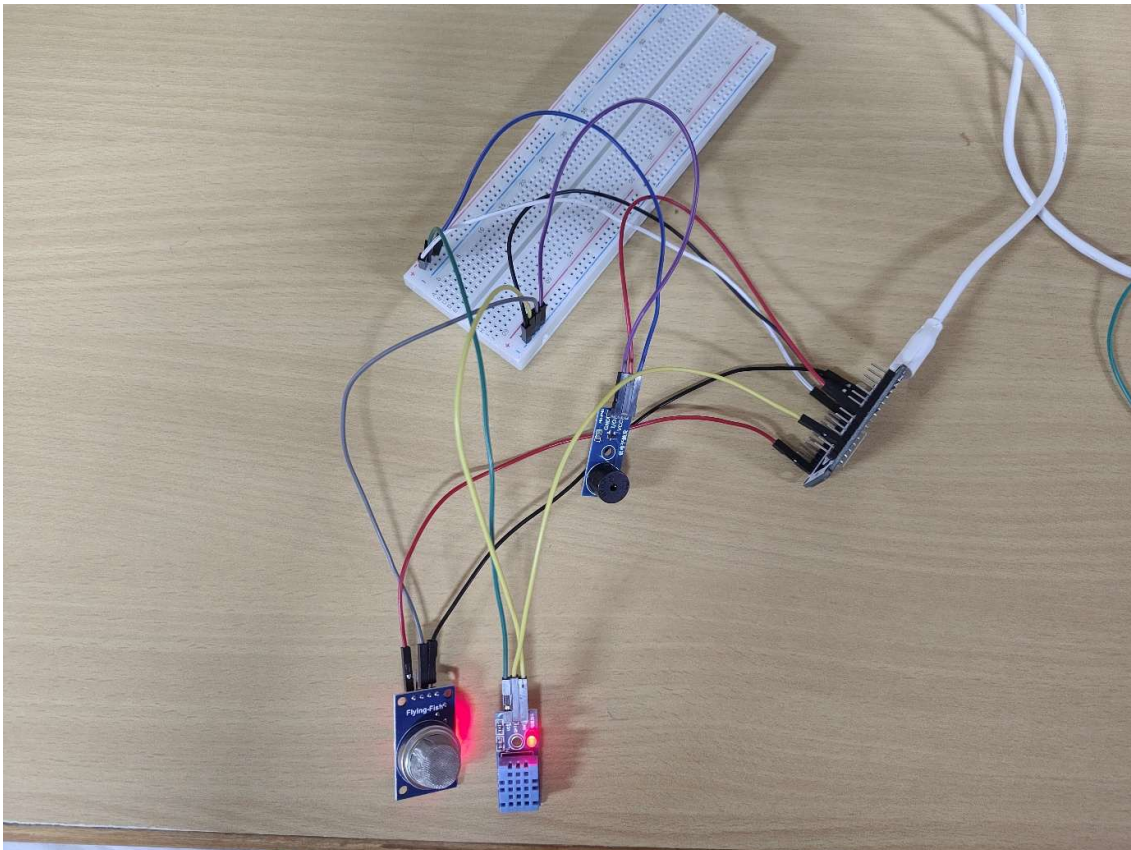
Input Section: DHT11/DHT22 and MQ135 sensors collect environmental parameters.

Processing Section: ESP8266 Node MCU processes and evaluates the sensor readings.

Communication Section: ESP8266 Wi-Fi enables wireless transmission of environmental data.

Alert Section: The buzzer provides an audible warning when abnormal conditions are detected.

Image:



Circuit Table

Component	Pin/Terminal	ESP8266 Node MCU Connection	Purpose
DHT11/DHT22	VCC	3.3V	Sensor power
	GND	GND	Common ground
	DATA	D4 (GPIO2)	Temperature & humidity data
MQ135	VCC	3.3V*	Sensor power
	GND	GND	Common ground
	AO	A0	Air-quality analog signal
Buzzer	+	D5 (GPIO14)	Alert output
	–	GND	Common ground

Important Pin Mapping:

- DHT11/DHT22 DATA → D4 (GPIO2)
- MQ135 AO → A0
- Buzzer Positive → D5 (GPIO14)
- DHT Sensor VCC → 3.3V
- MQ135 GND → GND
- Buzzer Negative → GND
- All components must have a common ground.

Algorithm

- **Step 1:** Start the system.
- **Step 2:** Initialize ESP8266, DHT sensor, MQ135 input, buzzer, and Wi-Fi.
- **Step 3:** Read temperature and humidity from the DHT sensor.
- **Step 4:** Read the analog output of the MQ135 sensor.
- **Step 5:** Process the sensor readings.
- **Step 6:** Compare the readings with predefined threshold values.
- **Step 7:** If the values are within the acceptable range, keep the buzzer OFF.
- **Step 8:** If an abnormal condition is detected, turn the buzzer ON.
- **Step 9:** Transmit the environmental data through Wi-Fi.
- **Step 10:** Repeat the monitoring process continuously.

Coding

```
#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

#include "DHT.h"

// --- Wi-Fi & ThingSpeak Credentials ---

const char* WIFI_SSID = "Rithick";      // Wi-Fi Name

const char* WIFI_PASS = "12345678";    // Wi-Fi Password


unsigned long channelID = 3459513;      // ThingSpeak Channel ID

const char* writeAPIKey = "CL0HI8YS4QH3QUIE"; // Write API Key


// --- Corrected Pin Definitions ---

#define DHTPIN    4    // GPIO 4 corresponds to physical pin D2 on
NodeMCU

#define DHTTYPE    DHT11 // DHT 11 sensor type

#define MQ2PIN    A0    // NodeMCU Analog Pin (A0)

#define BUZZER_PIN  5    // GPIO 5 corresponds to physical pin D1 on
NodeMCU


// --- Threshold Settings ---

#define GAS_THRESHOLD 350 // Adjust based on clean air baseline (0-
1023)

#define TEMP_THRESHOLD 40.0 // Alert temperature limit in °C
```

```
DHT dht(DHTPIN, DHTTYPE);

WiFiClient client;


// ThingSpeak minimum update interval is 15 seconds
unsigned long lastPublishTime = 0;

const unsigned long publishInterval = 15000;


void setup() {
  Serial.begin(115200);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);


  dht.begin();


  // Connect to Wi-Fi
  Serial.print("\nConnecting to Wi-Fi: ");

  Serial.println(WIFI_SSID);

  WiFi.begin(WIFI_SSID, WIFI_PASS);


  while (WiFi.status() != WL_CONNECTED) {
    delay(500);

    Serial.print(".");
```

```
}

Serial.println("\nWi-Fi Connected successfully!");

Serial.print("IP Address: ");

Serial.println(WiFi.localIP());


// Initialize ThingSpeak

ThingSpeak.begin(client);

}


void loop() {

    // Read Sensors

    float humidity = dht.readHumidity();

    float temperature = dht.readTemperature();

    int gasLevel = analogRead(MQ2PIN);


    // Check for DHT reading errors

    if (isnan(humidity) || isnan(temperature)) {

        Serial.println("Failed to read from DHT sensor! Checking wiring on pin D2...");

        delay(2000);

        return;

    }


    // Local Serial Monitor Print
```



```
Serial.print("Temp: ");

Serial.print(temperature);

Serial.print(" °C | Humidity: ");

Serial.print(humidity);

Serial.print(" % | Gas/Air Quality: ");

Serial.println(gasLevel);


// Alarm Logic (Runs in real-time)

if (gasLevel > GAS_THRESHOLD || temperature > TEMP_THRESHOLD) {

    Serial.println("ALARM TRIGGERED! Threshold Exceeded.");

    triggerBuzzerAlert();

} else {

    digitalWrite(BUZZER_PIN, LOW);

}


// Publish to ThingSpeak every 15 seconds

if (millis() - lastPublishTime >= publishInterval) {

    ThingSpeak.setField(1, temperature);

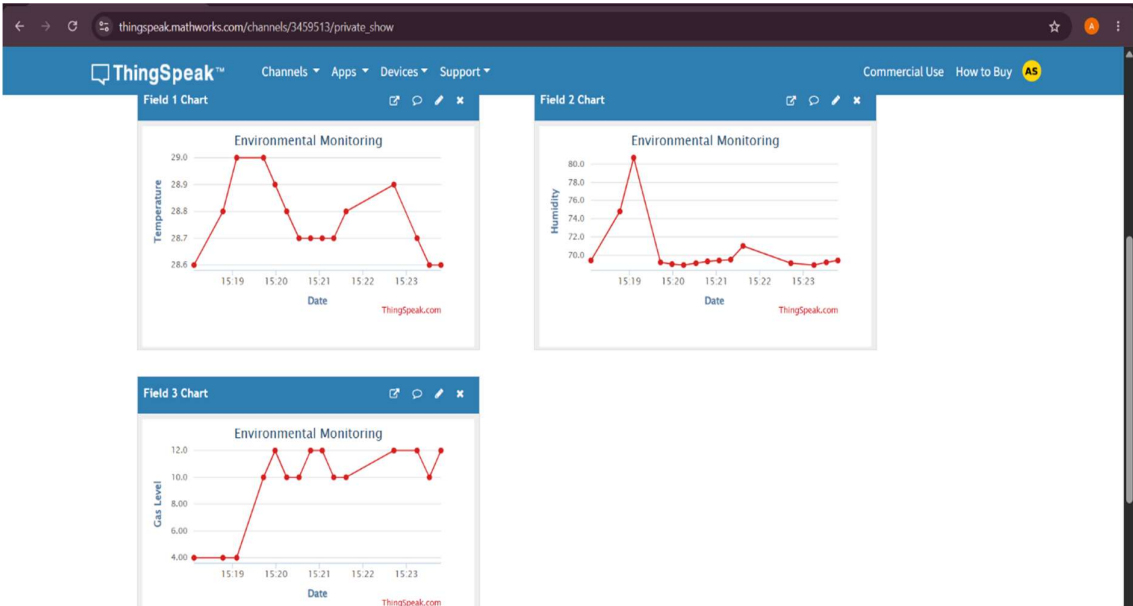
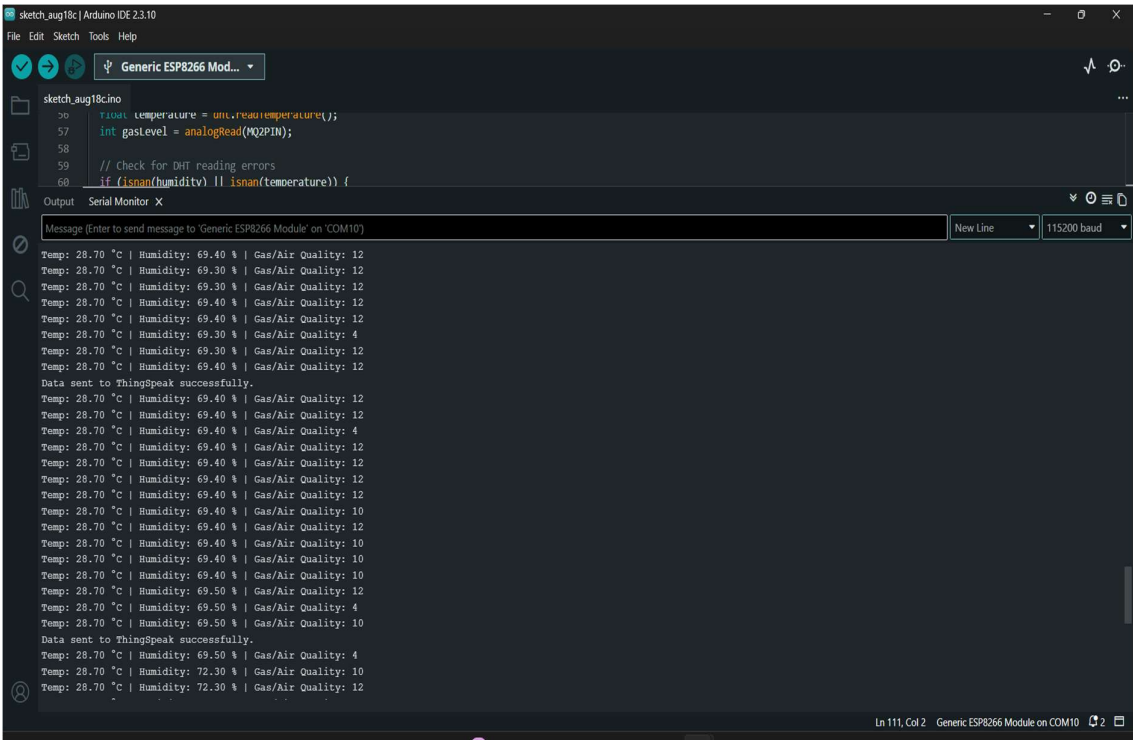
    ThingSpeak.setField(2, humidity);

    ThingSpeak.setField(3, gasLevel);


    int httpCode = ThingSpeak.writeFields(channelID, writeAPIKey);
```

```
    if (httpCode == 200) {  
        Serial.println("Data sent to ThingSpeak successfully.");  
    } else {  
        Serial.print("ThingSpeak Error. HTTP Code: ");  
        Serial.println(httpCode);  
    }  
  
    lastPublishTime = millis();  
}  
  
delay(1000);  
}  
  
// Sound Alarm Function for 3-Pin Active/Passive Buzzer  
void triggerBuzzerAlert() {  
    for (int i = 0; i < 2; i++) {  
        digitalWrite(BUZZER_PIN, HIGH);  
        delay(100);  
        digitalWrite(BUZZER_PIN, LOW);  
        delay(100);  
    }  
}
```

Coding Output:



System Working

- When the system is powered ON, the ESP8266 initializes all connected components.
- The DHT11/DHT22 senses the surrounding temperature and humidity.
- The MQ135 senses changes in air quality.
- The ESP8266 receives and processes these sensor readings.
- The readings are displayed through the Serial Monitor during prototype testing.
- The ESP8266 connects to the available Wi-Fi network.
- Environmental information can be transmitted wirelessly for remote monitoring.
- The controller compares the sensor values with the defined limits.
- When the environment is within the acceptable range, the buzzer remains OFF.
- When an abnormal condition is detected, the buzzer turns ON.
- The process repeats continuously to provide real-time monitoring.

Bill of Materials

S. No	Component	Quantity	Purpose
1	ESP8266 Node MCU	1	Main controller and Wi-Fi communication
2	DHT11/DHT22 Sensor	1	Temperature and humidity measurement
3	MQ135 Air Quality Sensor	1	Air-quality sensing
4	Buzzer	1	Audible warning
5	Breadboard	1	Circuit assembly
6	Jumper Wires	1 set	Electrical connections
7	USB Cable / 5 V Supply	1	Power and programming

Prototype Implementation

The prototype is implemented by connecting the DHT11/DHT22 sensor, MQ135 sensor, and buzzer to the ESP8266 Node MCU on a breadboard.

The DHT sensor is connected to a digital GPIO pin, while the MQ135 analog output is connected to the ESP8266 ADC input through appropriate voltage protection. The buzzer is connected to a digital output pin.

The ESP8266 is programmed using the Arduino IDE. After programming, it connects to a Wi-Fi network and continuously reads the environmental parameters. The sensor values are processed and abnormal conditions trigger the buzzer.

Hardware Setup:

Setup	Connection
ESP8266 Node MCU	Main controller
DHT11/DHT22	Connected to D4
MQ135	Analog output connected to A0 with suitable voltage protection
Buzzer	Connected to D5
Breadboard	Used for mounting components
Jumper wires	Used for electrical connections
Power	USB connection to Node MCU

Hardware Assembly:

- Place the ESP8266 Node MCU on the breadboard.
- Connect the DHT11/DHT22 sensor to the Node MCU.
- Connect the MQ135 sensor module.
- Ensure the MQ135 analog output is within the safe ADC input range.
- Connect the buzzer to the digital output.
- Connect all required grounds together.
- Power the Node MCU through USB.
- Upload the program using Arduino IDE.
- Connect the system to Wi-Fi.

Prototype Working

After powering the prototype, the DHT11/DHT22 starts measuring temperature and humidity, while the MQ135 continuously detects changes in air quality.

The ESP8266 processes the sensor readings and sends them through its Wi-Fi interface. During testing, the readings can be observed through the Arduino Serial Monitor.

When the sensor values remain within the defined limits, the buzzer remains OFF. When an environmental parameter crosses its threshold, the ESP8266 activates the buzzer to indicate a potentially abnormal condition.

Expected Prototype Output

Parameter	Normal Condition	Abnormal Condition	System Response
Temperature	Within set limit	Above set limit	Buzzer ON
Humidity	Within set limit	Above set limit	Buzzer ON
Air Quality	Acceptable sensor reading	Above calibrated threshold	Buzzer ON
All Parameters	Normal	—	Buzzer OFF

Prototype Demonstration

During the demonstration, the system is powered through the ESP8266 USB connection. The DHT11/DHT22 continuously measures temperature and humidity, while the MQ135 monitors air-quality changes.

The sensor readings are observed through the Serial Monitor. A controlled environmental change can be introduced during testing to verify the response of the sensors. When a programmed threshold is exceeded, the ESP8266 activates the buzzer.

The Wi-Fi functionality demonstrates the wireless capability of the system and can be extended to a web dashboard or IoT cloud platform for remote monitoring.

Results & Performance Analysis

The prototype successfully demonstrates wireless monitoring of important environmental parameters using low-cost sensors and an ESP8266 controller.

Parameter	Observation	Performance
Temperature	Continuously measured by DHT sensor	Good
Humidity	Continuously measured by DHT sensor	Good
Air Quality	Changes detected using MQ135	Satisfactory
Wireless Communication	ESP8266 Wi-Fi connection	Good
Alert System	Buzzer activated during threshold violation	Good
Continuous Monitoring	Repeated sensor measurements	Good

Expected Results:

- Continuous measurement of temperature.
- Continuous measurement of humidity.
- Detection of changes in air quality.
- Wireless transmission capability through ESP8266.
- Immediate buzzer alert during abnormal conditions.
- Reduced dependence on manual environmental inspection.
- Improved awareness of industrial environmental conditions.
- Low-cost and compact prototype suitable for smart-industry applications.

Performance Analysis

Performance Parameter	Expected Performance
Temperature Monitoring	Continuous
Humidity Monitoring	Continuous
Air Quality Monitoring	Continuous
Wireless Connectivity	Wi-Fi based
Alert Response	Immediate after threshold detection
Monitoring Method	Real-time
Controller	ESP8266 Node MCU
System Cost	Low
Expandability	High
Maintenance	Low